FINAL PROJECT REPORT

Financial Fraud Detection

A Risk-Flag Model and Power BI Dashboard

Author:	Akash Vaishnav
Role:	Data Science Intern
Company:	Zidio Development
Mentor:	Chandan Mishra
Date:	October 2026
Tools:	Python (JupyterLab, pandas, scikit-learn, XGBoost) and Power BI

Acknowledgment

I would like to thank Zidio Development for giving me the opportunity to work on this project during my internship. It helped me practice the full data science workflow, from choosing a dataset to building a model and a dashboard.

I am especially grateful to my mentor, Chandan Mishra, for his guidance and feedback during the project.

This project also taught me to check my results honestly. The first dataset I chose had no usable signal, so I changed it, and this report states the limits of the final model clearly.

Abstract

This project builds a model that finds fraudulent card and online transactions and shows the results in a Power BI dashboard. Four datasets were tested. Three of them were rejected because they contained no usable pattern or contained a leaking column. The chosen dataset has 5,000 transactions, of which 9.64% are fraud. Exploratory analysis showed that only three transaction features separate fraud from genuine transactions: a suspicious keyword, an international transaction and a night-time transaction (hours 0 to 5). All other fields showed no link to fraud.

Logistic Regression, Random Forest and XGBoost gave the same cross-validation scores (ROC-AUC about 0.89 to 0.90), so the simplest model was chosen: Logistic Regression with balanced class weights on the three risk flags. On the unseen test set it reached a ROC-AUC of 0.907 and a PR-AUC of 0.453. It caught all 120 test frauds, but it also raised 324 false alarms, so its precision is only 27%. The report explains why this limit comes from the data and not from the algorithm, and it lists the other limits of the project, including that the data is synthetic.

Contents

Acknowledgment 2
Abstract 2
1. Introduction 4
1.1 Background 4
1.2 Problem statement 4
1.3 Objectives 4
1.4 Scope and tools 4
2. Dataset Description 5
2.1 Datasets tested 5
2.2 Main dataset 5
2.3 Why the dataset was changed 6
3. Data Cleaning and Feature Engineering 6
3.1 Cleaning steps 6
3.2 Feature engineering 7
3.3 Features tested and dropped 7
4. Exploratory Data Analysis 8
4.1 Class imbalance 8
4.2 Three strong signals 8
4.3 Combining the flags 8
4.4 Fields with no signal 9
5. Modelling and Evaluation 10
5.1 Experimental setup 10
5.2 Model comparison 10
5.3 Feature importance and feature ablation 11
5.4 Handling the class imbalance 12
5.5 Threshold selection and the business trade-off 12
5.6 Final model and test results 13
6. Power BI Dashboard 14
6.1 Purpose and design 14
6.2 Introduction page 15
6.3 Page 1: Fraud Overview 16
6.4 Page 2: Model Performance 17
6.5 Page 3: Features That Do Not Matter 18
6.6 Technical notes 19
7. Limitations and Conclusion 19
7.1 Limitations 19
7.2 Conclusion 19
8. Future Work 20
Appendix A: Project Files 20
Appendix B: Streamlit App 21

1. Introduction

1.1 Background

Financial fraud is a serious problem for banks and payment companies. They process a very large number of transactions every day, and only a small share of them is fraudulent. Analysts cannot check every transaction by hand, so companies use data-driven models to find the transactions that look risky and send those for review first.

A good fraud model has to do two things at the same time. It must catch as many real frauds as possible, and it must avoid raising too many false alarms, because every alert costs analyst time.

1.2 Problem statement

Given the details of a transaction, can we predict whether it is fraudulent, and can we present the result in a way that a business user can understand and use?

1.3 Objectives

- Find a dataset that really contains a fraud pattern, and check its quality.
- Clean the data, explore it, and create useful features.
- Compare several models and evaluate them honestly on data they did not see during training.
- Handle the class imbalance (only 9.64% of the transactions are fraud).
- Choose a decision threshold and explain the business trade-off.
- Build a Power BI dashboard for a business audience.
- Document the limits of the work clearly.

1.4 Scope and tools

This is an internship and portfolio project. The analysis uses Python (JupyterLab, pandas, scikit-learn and XGBoost), and the dashboard uses Power BI Desktop. The final dataset is synthetic (computer-generated). The results therefore show a sound method and an honest evaluation, but they do not show how the model would perform on real bank data.

2. Dataset Description

2.1 Datasets tested

Before modelling, four datasets were checked. A model can only learn a pattern that exists in the data. If the best model scores like a coin flip, any good result would come from noise or from leakage. The measure used for this check is ROC-AUC: a value of 0.50 means random guessing and 1.0 means a perfect ranking.

Dataset	Rows	What was found	Decision
synthetic_fraud_dataset1.csv (target **Fraud_Label**, 32.1% fraud)	50,000	No link between any feature and the target. Chi-square p-values were 0.34 to 0.74 for all categorical features, the numeric means were equal for fraud and non-fraud, and a baseline Random Forest gave ROC-AUC 0.51.	Rejected
credit_card_fraud_dataset.csv	100,000	ROC-AUC about 0.50, so no signal.	Rejected
Fraud_Detection.xlsx	n/a	Contains a leaking column named **isFraud - Copy** (AUC 1.0) and only 68 fraud rows.	Rejected
financial_fraud_detection_dataset.csv (target **Fraudulent**, 9.64% fraud)	5,000	Clear and explainable patterns in three features.	Used

The first dataset was the largest, so it looked attractive, but it was the clearest example of a dataset with no signal. The **Fraud_Detection.xlsx** file shows another danger. A column that copies the target gives a perfect score, but such a model is useless in real life because the answer is already inside the input (data leakage). Keeping these rejected datasets as a finding in the report shows that the final dataset was chosen with evidence and not by accident.

2.2 Main dataset

The main dataset is **financial_fraud_detection_dataset.csv**. It has 5,000 rows and 14 columns, with no missing values and no duplicate rows. The target column is **Fraudulent**: 90.36% of the transactions are genuine and 9.64% are fraud (482 transactions). The dates run from 1 January 2023 to 21 February 2024.

Column	Description
Transaction_ID	Unique identifier of the transaction
Customer_ID	Customer identifier (3,847 unique values in 5,000 rows)
Transaction_Date	Date and time of the transaction (format dd-mm-yyyy hh:mm)
Transaction_Amount	Amount of the transaction
Merchant_Category	Electronics, Entertainment, Fashion, Food, Grocery, Health, Travel, Utilities
Payment_Method	Credit Card, Debit Card, NetBanking, PayPal, UPI
Device_Type	Desktop, Mobile, POS
Location	Seven Indian cities: Bengaluru, Chennai, Delhi, Hyderabad, Kolkata, Mumbai, Pune

Column	Description
Is_International	1 if the transaction is international, otherwise 0
Previous_Transactions	Number of previous transactions of the customer
Average_Spend	Average spend of the customer
Account_Age_Days	Age of the customer account in days
Suspicious_Keyword	Yes or No: whether a suspicious keyword was present
Fraudulent	Target: 1 for fraud, 0 for genuine

2.3 Why the dataset was changed

The project started with **synthetic_fraud_dataset1.csv**. During exploratory analysis every feature looked the same for fraud and non-fraud transactions, and a baseline Random Forest reached only 0.51 ROC-AUC. Spending more time on models would not have helped, because there was nothing to learn. The dataset was therefore dropped and the search continued until a dataset with a real pattern was found.

> **Honest note:** the final dataset is also synthetic. It has clear patterns, but real fraud is usually less clean. All results in this report should be read with this in mind.

3. Data Cleaning and Feature Engineering

3.1 Cleaning steps

- **Quality checks:** no missing values and no duplicate rows were found.
- **Dates: Transaction_Date** was converted from text to a date-time column using the format **%d-%m-%Y %H:%M**.
- **Identifiers: Customer_ID** repeats (3,847 unique values), but the profile values of the same customer change in every row, so the identifier carries no information. **Customer_ID** and **Transaction_ID** were dropped.
- **Zero amounts:** two transactions have an amount of zero. They were kept and noted.
- **Skewed amount: Transaction_Amount** has a skewness of 1.9, so a log version was created.

3.2 Feature engineering

New feature	How it was created	Why
Hour	Hour taken from **Transaction_Date**	Fraud may depend on the time of day
Is_Night	1 if the hour is between 0 and 5, otherwise 0	Night transactions showed a high fraud rate
Suspicious_Keyword (0/1)	Yes mapped to 1 and No mapped to 0	Numeric form for the models
Log_Amount	log(1 + **Transaction_Amount**)	Reduces the skew of the amount
Weekday, **Month**, **Is_Weekend**	Taken from **Transaction_Date**	Tested for calendar effects
Amount / **Average_Spend** ratio	Amount divided by the customer's average spend	Tested for unusual spending
Risk_Flags	Count of the three flags (night, international, keyword)	Used in the analysis and the dashboard

3.3 Features tested and dropped

Every feature was tested before it was allowed into the model. The table shows the result.

Feature(s)	Test result	Decision
Is_Night, **Is_International**, **Suspicious_Keyword**	Large differences in fraud rate between groups	**Kept**
Transaction_Amount, **Average_Spend**, **Previous_Transactions**, **Account_Age_Days**	Means are almost equal for fraud and non-fraud	Dropped from final model
Payment_Method, **Device_Type**, **Location**, **Merchant_Category**	Chi-square p-values 0.31 to 0.62	Dropped
Weekday, **Month**, **Is_Weekend**	p-values 0.26, 0.54 and 0.65	Dropped
Amount / **Average_Spend** ratio	No signal	Dropped
Customer_ID, **Transaction_ID**	Identifiers with no information	Dropped

> **Caution:** the night cutoff (hours 0 to 5) and the choice of the three flags were decided after looking at the full dataset, including the rows that were later used for testing. This is mild data snooping, so the test result is slightly optimistic. See Section 7.

4. Exploratory Data Analysis

4.1 Class imbalance

Only 482 of the 5,000 transactions (9.64%) are fraud. This matters for two reasons. First, accuracy is a misleading measure: a model that always answers "genuine" would be 90.4% accurate and would catch no fraud at all. Second, the training must pay attention to the small fraud class.

4.2 Three strong signals

Three fields clearly separate fraud from genuine transactions:

- **Suspicious keyword:** 47.3% fraud when a keyword is present (260 rows) against 7.6% when it is not.
- **International transaction:** 37% fraud for international transactions (441 rows) against 7% for domestic ones.
- **Night hours (0 to 5):** 25.6% fraud (1,222 rows) against 4.3% to 4.8% in the other time blocks.

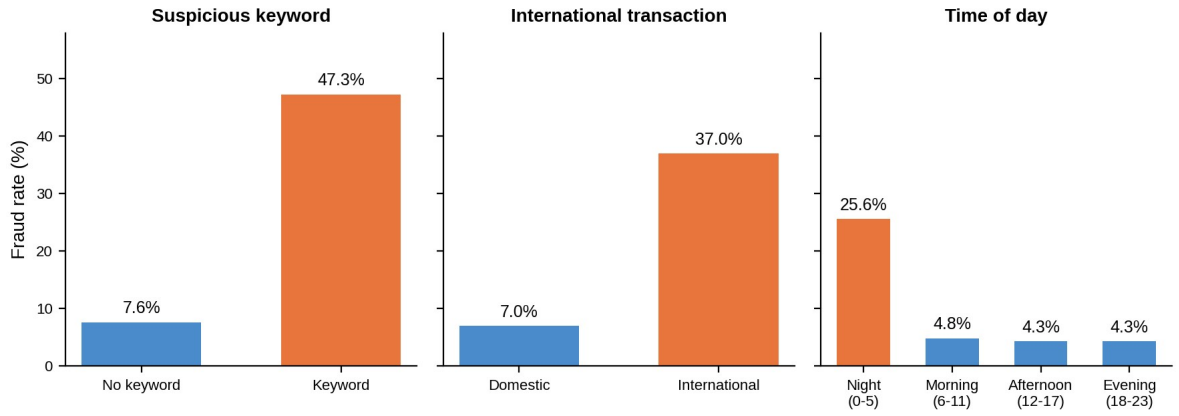


Figure 1. Fraud rate for the three strong signals (all 5,000 transactions).

4.3 Combining the flags

The three signals add up. The more risk flags a transaction has, the more likely it is fraud.

Risk flags	Transactions	Frauds	Fraud rate
0	3,278	1	0.03%
1	1,528	370	24.2%
2	187	104	55.6%
3	7	7	100%

481 of the 482 frauds have at least one flag. This explains why a model based on these flags can find almost every fraud, and it also hints at why it cannot be very precise: 1,528 transactions with exactly one flag include 370 frauds and 1,158 genuine transactions.

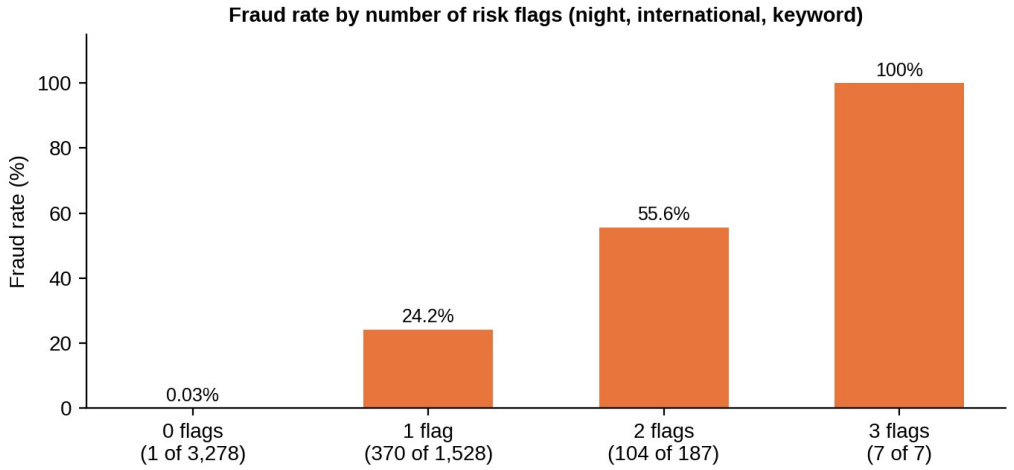


Figure 2. Fraud rate by number of risk flags. The 3-flag group has only 7 transactions.

4.4 Fields with no signal

The numeric columns (amount, average spend, previous transactions and account age) have almost the same averages for fraud and genuine transactions. Payment method, device type, location and merchant category also show no link to fraud (chi-square p-values 0.31 to 0.62), and so do weekday, month and weekend (p-values 0.26, 0.54 and 0.65). Figure 3 compares the fields with a signal and the fields without a signal. The small differences between groups of the blue fields are what we expect from chance, because each group has only a few hundred rows.

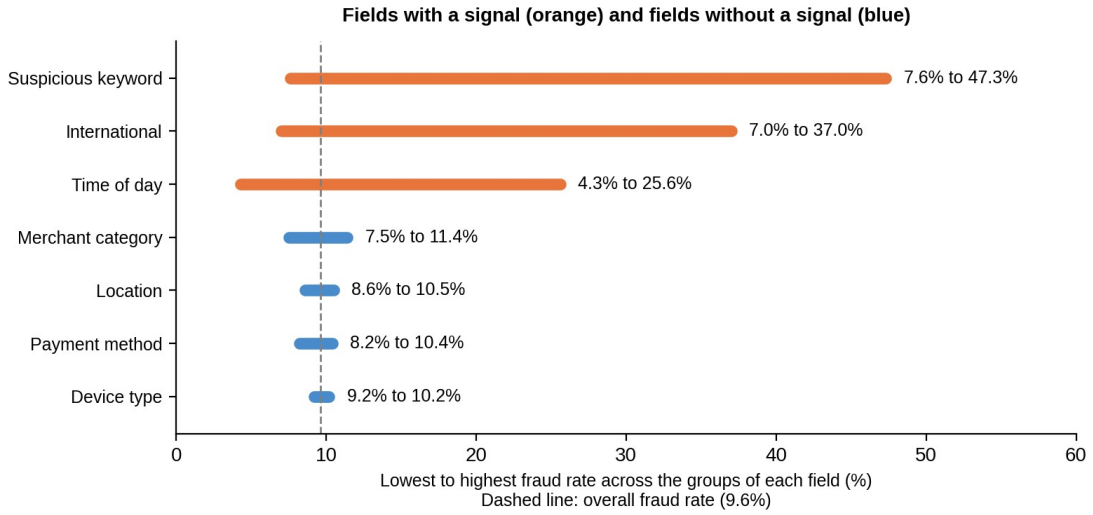


Figure 3. Range of fraud rate across the groups of each field. Values for the four blue fields are read from dashboard page 3.

5. Modelling and Evaluation

5.1 Experimental setup

The data was split into a training set (75%) and a test set (25%) with stratification, so both parts keep the same fraud rate. The random state was 42. This gave 3,750 training rows (fraud rate 9.65%) and 1,250 test rows (fraud rate 9.6%). The test set was kept aside and was used only for the final result. All choices were made with 5-fold cross-validation on the training set.

Because the classes are imbalanced, the following measures were used instead of accuracy:

- **ROC-AUC:** how well the model ranks frauds above genuine transactions (0.5 is random, 1.0 is perfect).
- **PR-AUC:** the same idea, but focused on the fraud class. Its random baseline is the fraud rate, about 0.096.
- **Precision:** of the transactions flagged as fraud, the share that are really fraud.
- **Recall:** of all real frauds, the share that the model caught.
- **F1 score:** one number that combines precision and recall.

5.2 Model comparison

Logistic Regression, Random Forest and XGBoost were compared on the seven candidate features (**Is_Night**, **Is_International**, **Suspicious_Keyword**, **Log_Amount**, **Average_Spend**, **Previous_Transactions** and **Account_Age_Days**). A dummy model that ignores the features was used as a baseline (ROC-AUC 0.50, PR-AUC 0.096).

Model (5-fold CV on training set)	ROC-AUC	PR-AUC (mean +/- std)
Logistic Regression	0.902	0.447 +/- 0.042
Random Forest	0.897	0.410 +/- 0.051
XGBoost	0.892	0.388 +/- 0.034

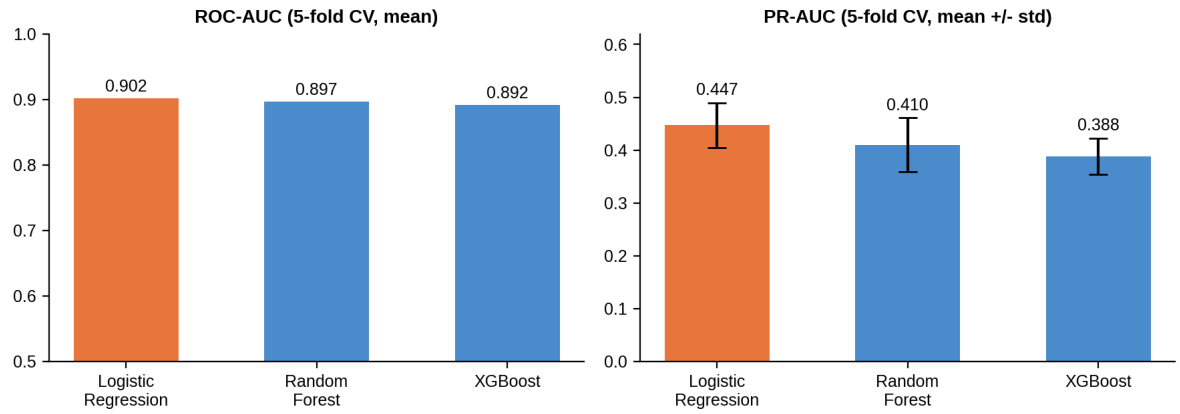


Figure 4. Cross-validation scores. The error bars overlap, so the models are tied.

All three models are tied within the error. A more complex model gives no benefit here, so the simplest and most explainable model, Logistic Regression, was chosen. The same pattern appeared on the test set at threshold 0.5:

Model (test set, 7 features)	Precision	Recall	F1	ROC-AUC	PR-AUC
Dummy baseline	-	-	-	0.500	0.096
Logistic Regression (balanced)	0.27	1.00	0.426	0.891	0.410
Random Forest (balanced)	0.34	0.575	0.427	0.886	0.387

The two models have the same F1 score (about 0.43), but they reach it in different ways: Logistic Regression catches every fraud with many false alarms, and Random Forest raises fewer alerts but misses 42% of the frauds.

5.3 Feature importance and feature ablation

Permutation importance on the test set (drop in PR-AUC for the Random Forest) and the scaled Logistic Regression coefficients point to the same three features:

Feature	Permutation importance	Comment
Suspicious_Keyword	0.147	Important
Is_International	0.119	Important
Is_Night	0.090	Important
Previous_Transactions	0.028	Probably noise
Log_Amount	0.003	No effect
Account_Age_Days	-0.001	No effect
Average_Spend	-0.013	No effect

In Logistic Regression the scaled coefficients were 1.884 for **Is_Night**, 1.301 for **Is_International** and 1.045 for **Suspicious_Keyword**, while all other coefficients were close to zero.

An ablation test confirmed this. With all seven features the cross-validated PR-AUC was 0.447 for Logistic Regression and 0.410 for Random Forest. With only the three flags it was 0.413 (+/- 0.023) and 0.393 (+/- 0.024), and ROC-AUC stayed between 0.897 and 0.902 in every case. The differences are within the error, so the numeric features add nothing that we can prove. The three-feature model was chosen because it is simpler and easier to explain, not because the seven-feature model was proven worse.

5.4 Handling the class imbalance

Three options were compared inside the cross-validation pipeline: class weights, no balancing, and SMOTE (which creates synthetic fraud examples). SMOTE was applied only to the training part of each fold, so no synthetic data leaked into the validation data.

Option	Result
Class weights vs no balancing	The ranking quality (ROC-AUC, PR-AUC) is the same. Balancing only changes which cases cross the threshold.
Logistic Regression + SMOTE	Same as class weights (PR-AUC 0.444)
Random Forest + SMOTE	Worse (ROC-AUC 0.805, PR-AUC 0.263). Most features are binary flags, so SMOTE creates unrealistic points.

Decision: class weights were used and SMOTE was not.

5.5 Threshold selection and the business trade-off

The threshold decides how many transactions are flagged. It was studied with out-of-fold predictions on the training set, using the three-flag Logistic Regression. The model gives scores that form three clusters: about 0.03 for no flag, about 0.7 for one flag and about 0.99 for two or three flags. Therefore, any threshold between about 0.03 and 0.68 means the same rule: flag a transaction if it has at least one risk flag. A higher threshold flags fewer transactions, those whose flag combination gives a higher score.

Rule (training set, 3,750 rows)	Flagged	Precision	Recall
Flag if at least one risk flag (threshold between 0.03 and 0.68)	1,278	0.282	0.997
Threshold about 0.725	500	0.386	0.533
Threshold about 0.788	146	0.589	0.238

The F2 score, which values recall more than precision, was best (0.662) for the first rule, so the final threshold is 0.5, which gives this rule. This choice rests on an assumption: a missed fraud costs more than a false alarm. The real costs were not available. If a bank has a very small review team, the stricter rules in the table may be a better choice.

5.6 Final model and test results

Final model: Logistic Regression with **class_weight="balanced"** and a **StandardScaler** in a scikit-learn pipeline, using three features (**Is_Night**, **Is_International**, **Suspicious_Keyword**) and a threshold of 0.5. The model was saved as **final_model.joblib**.

Test-set measure (1,250 transactions)	Value
ROC-AUC	0.907
PR-AUC	0.453
Precision (fraud class)	0.27
Recall (fraud class)	1.00
F1 score (fraud class)	0.43
Accuracy	0.74
Alerts per fraud caught (444 / 120)	3.7

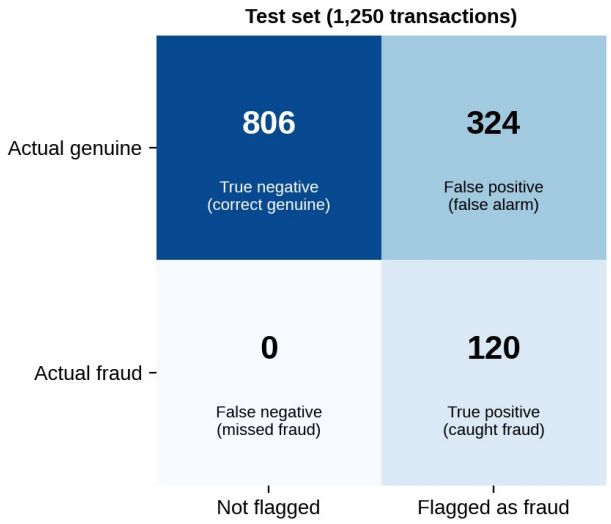


Figure 5. Confusion matrix on the test set: 806 correct genuine, 324 false alarms, 0 missed frauds, 120 caught frauds.

On all 5,000 rows (training and test together) the model gives 3,277 correct genuine transactions, 1,241 false alarms, 481 caught frauds and 1 missed fraud.

How to read this result. The model is very good at not missing fraud, but it is not precise: about three of every four alerts are false alarms. The ceiling of about 0.43 F1 score comes from the data. After the three flags there is almost no further information, so no algorithm can separate the genuine transactions that carry a flag from the real frauds that carry the same flag.

6. Power BI Dashboard

6.1 Purpose and design

The dashboard presents the results to a business reader who does not read code. It uses the file **dashboard_data.csv** (5,000 rows, 22 columns), which contains the original fields, the model probability, the predicted label and an outcome column (correct genuine, false alarm, caught fraud, missed fraud). The following design rules were used on every page:

- Charts start at zero, so small differences are not made to look big.
- The risky group is highlighted in orange and the normal groups in blue.
- Every page has a note that says which rows it uses (all 5,000 rows or the test rows only).
- Charts that are compared with each other use the same axis scale.

6.2 Introduction page



Figure 6. Introduction page of the dashboard.

6.3 Page 1: Fraud Overview

This page uses all 5,000 transactions. Nine cards show the main numbers: total transactions (5,000), fraud count (482), fraud rate (9.6%), flagged (1,722), caught fraud (481), false alarms (1,241), recall (99.8%), precision (27.9%) and alerts per fraud caught (3.58). Five charts show the fraud rate by time of day, by suspicious keyword, by number of risk flags, by international status and by month. Three slicers (location, merchant category and payment method) let the reader filter the page. The monthly fraud rate moves between about 8% and 12.7%, but with about 350 transactions per month this is normal random variation and not a trend.

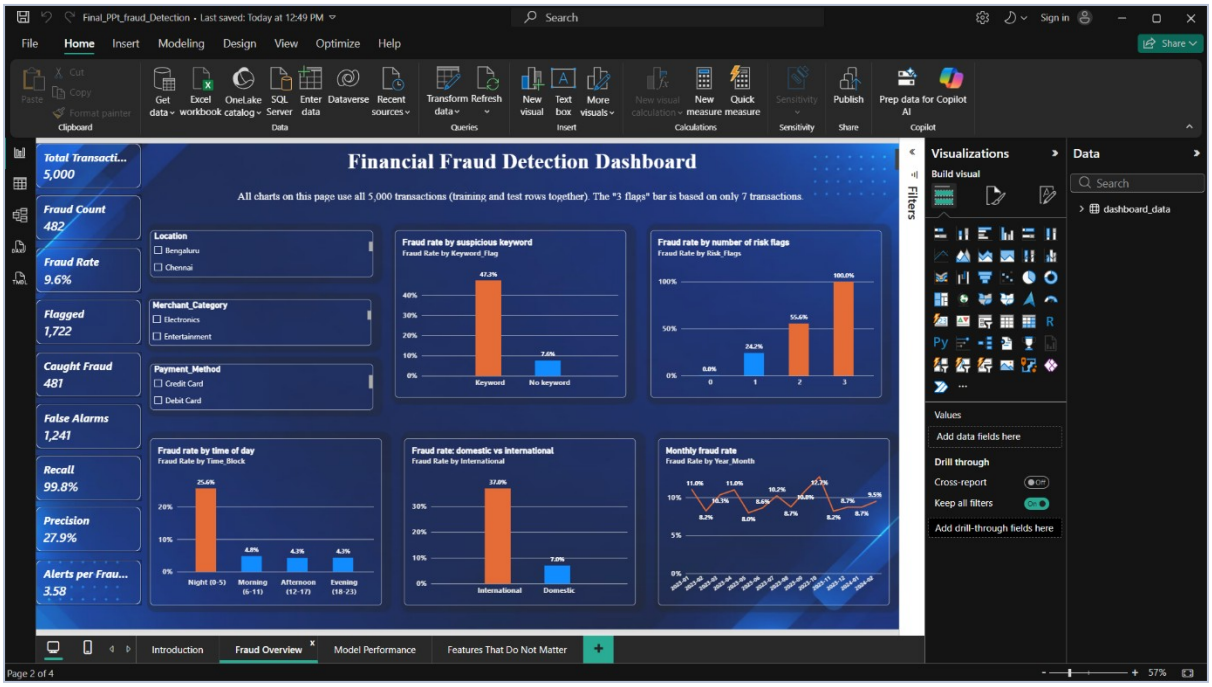


Figure 7. Dashboard page 1: Fraud Overview.

6.4 Page 2: Model Performance

This page uses only the 1,250 test transactions, because the model must be judged on data it did not see during training. The cards show 120 frauds, 444 flagged transactions, 120 caught frauds, 324 false alarms, recall 100.0%, precision 27.0% and 3.70 alerts per fraud caught. A matrix and a bar chart show the confusion matrix, and a note warns that 100% recall comes from only 120 frauds.

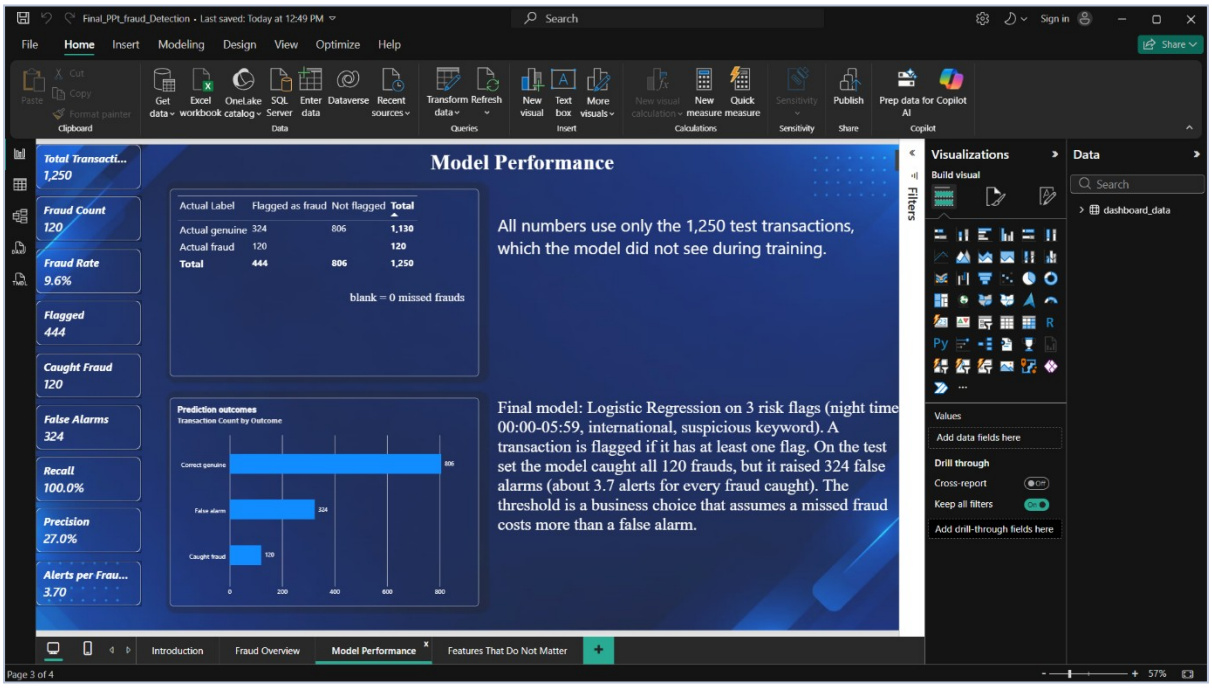


Figure 8. Dashboard page 2: Model Performance (test rows only).

6.5 Page 3: Features That Do Not Matter

This page uses all 5,000 transactions. Four charts (location, payment method, device type and merchant category) use the same 0 to 30% scale. The bars are almost equal, and the fraud rate of every group stays close to the overall 9.6% (between 7.5% and 11.4%). This is the visual proof of why these fields were dropped from the model.

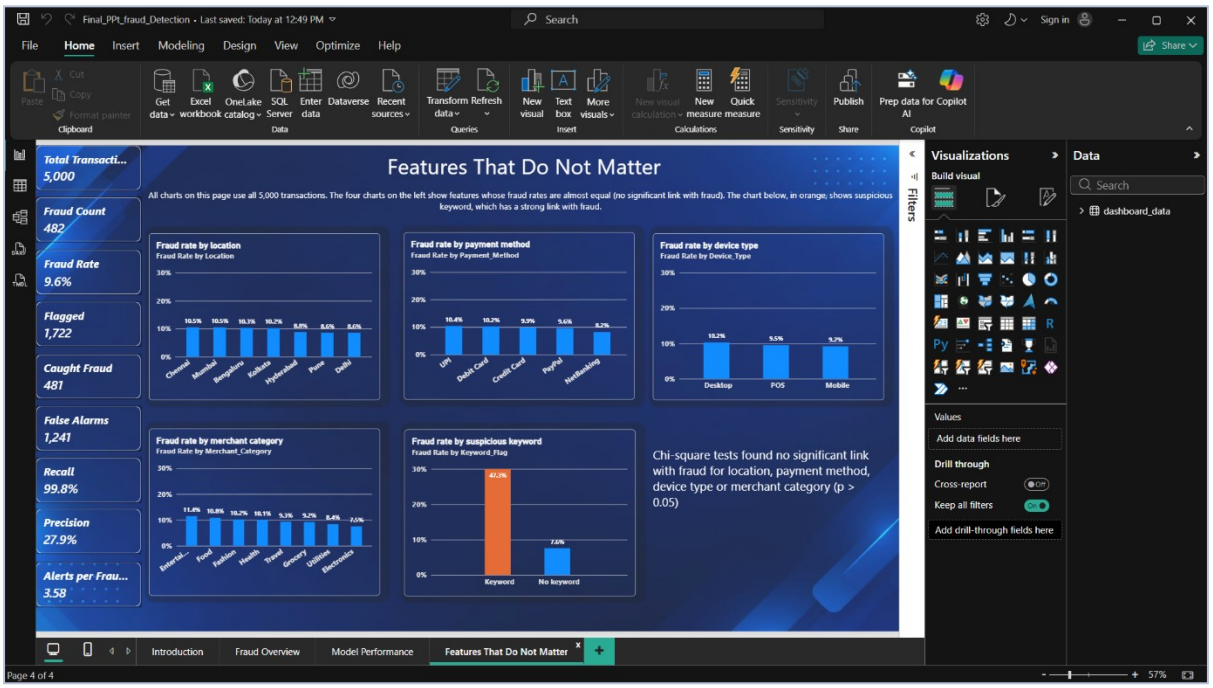


Figure 9. Dashboard page 3: Features That Do Not Matter.

6.6 Technical notes

- Measures such as **Total Transactions**, **Fraud Count**, **Fraud Rate**, **Flagged**, **Caught Fraud**, **False Alarms**, **Recall**, **Precision** and **Alerts per Fraud Caught** were written in DAX.
- The cards show text versions of the measures so that the numbers appear in full (for example 5,000 and 9.6%), while the charts use the numeric measures.
- The file is saved as **fraud_dashboard.pbix** in the dashboard folder.

7. Limitations and Conclusion

7.1 Limitations

The following limits should be kept in mind when reading the results.

Limitation	What it means
Low precision (27%)	About three of every four alerts are false alarms. The model needs 3.7 alerts to find one fraud. In real use this costs analyst time.
Small dataset	There are only 5,000 transactions and 482 frauds. The test set has only 120 frauds, so every test metric is uncertain. Cross-validation scores vary by about 0.04 to 0.05.
Synthetic data	The patterns are probably simple rules built into the data generator. Real fraud is more complex, so the numbers must not be used as a promise of real-world performance.
Data snooping	The three flags and the night cutoff (hours 0 to 5) were chosen after looking at the full dataset, including the test rows. The test result is therefore slightly optimistic.
F1 ceiling of about 0.43	All models reach the same score. The limit comes from the data, because only three features carry information.
The model is a simple rule	At threshold 0.5 the model is the same as the rule "flag a transaction if it has at least one risk flag". Machine learning adds little beyond this rule on this dataset.
Recall of 100%	On the test set the recall is 100% (120 of 120). In cross-validation it was 99.7%, and across all 5,000 rows the model missed 1 fraud. It is not a perfect model.
Very small groups	The 3-flag group has only 7 transactions, so its 100% fraud rate is not a reliable rule.
Assumed costs	The threshold assumes that a missed fraud costs more than a false alarm. The real costs were not known.
Random split	The data was split at random. A split by time was not tested, and real systems must work on future data.

7.2 Conclusion

This project followed the complete data science workflow: choosing a dataset with evidence, cleaning and exploring it, testing every feature, comparing models, handling the class imbalance, choosing a threshold and presenting the results in a dashboard.

The main findings are clear. Only three features (a suspicious keyword, an international transaction and a night-time transaction) are linked to fraud, and three different models reach the same performance on them. The final Logistic Regression model catches all frauds in the

test set (recall 1.00) with a ROC-AUC of 0.907, but its precision is only 0.27. The most useful lesson of the project is that the quality of the data, not the choice of algorithm, limits the result, and that honest reporting of this limit is part of good data science.

8. Future Work

- **Real or realistic data:** test the approach on anonymized real transactions, where the patterns are less clean.
- **Time-based validation:** train on older transactions and test on newer ones, as in real use.
- **Cost-based threshold:** replace the assumption with the real cost of a missed fraud and of a false alarm, and choose the threshold that minimizes the total cost.
- **Richer features:** add behavior features such as the number of transactions in the last hour, the distance from the usual location, the first use of a new device or a new merchant.
- **Probability calibration:** check that a score of 0.7 really means about 70% risk, so the scores can be used for ranking and prioritizing alerts.
- **Explainability:** add explanations for each alert, for example with SHAP, so analysts can see why a transaction was flagged.
- **Deployment and monitoring:** serve the model through an API, and monitor its precision, recall and data drift over time.
- **Dashboard:** refresh the dashboard automatically and add a view of alerts per day for the review team.

Appendix A: Project Files

File or folder	Content
data\financial_fraud_detection_dataset.csv	Main dataset (5,000 rows, 14 columns)
notebooks\01_fraud_detection.ipynb	Full analysis and modelling notebook
final_model.joblib	Saved final model (Logistic Regression pipeline)
dashboard_data.csv	Data exported for Power BI (5,000 rows, 22 columns)
dashboard\fraud_dashboard.pbix	Power BI dashboard
streamlit_app\app.py	Streamlit app for live testing (see Appendix B)

Appendix B: Streamlit App

B.1 Purpose

Besides the Power BI dashboard, the final model is also available as a small web app called **Financial Fraud Risk Checker**. It lets a reader test the model without reading code. The reader enters three details of a transaction, and the app shows the fraud probability and tells whether the transaction would be sent for review. The app is public and opens in any web browser.

Live app: https://financial-fraud-risk-checker.streamlit.app/

Code: github.com/akashvai8931-stack/financial-fraud-detection (folder streamlit_app)

B.2 How the app works

The app asks for three inputs. They are turned into the same three risk flags that the final model uses (Section 5.6).

Input in the app	Possible values	Used by the model as
Hour of the transaction	Slider from 0 to 23	Is_Night: 1 if the hour is 0 to 5, otherwise 0
Is the transaction international?	No or Yes	Is_International: 0 or 1
Was a suspicious keyword present?	No or Yes	Suspicious_Keyword: 0 or 1

When the reader clicks **Check transaction**, the app loads the saved model (final_model.joblib) and shows the fraud probability, the number of risk flags (0 to 3), a message that says whether the transaction is flagged for review, the list of flags that are present, and the fraud rate seen in the 5,000-row dataset for the same number of flags. A box called “About this model and its limits” explains the limits of the model.



Figure 10. The Streamlit app when it opens.

B.3 Example result



Figure 11. Result for a transaction at 2 o'clock at night, international, with a suspicious keyword: probability 100.0%, 3 of 3 risk flags, flagged for review.

The model uses only three yes/no flags, so the app can give only a few different answers. The table shows what to expect for each number of flags. A transaction with at least one flag is flagged, which is the same rule as in Section 5.5.

Risk flags	Model probability (about)	Decision at threshold 0.5	Fraud rate in the 5,000 rows
0	0.03	Not flagged	0.03% (1 of 3,278)
1	0.7	Flagged	24.2% (370 of 1,528)
2	0.99	Flagged	55.6% (104 of 187)
3	1.00	Flagged	100% (7 of 7)

B.4 Limits of the app

- **Same limits as the model:** the precision is 0.27 on the test set, so about three of every four alerts are false alarms (Section 5.6).
- **Synthetic data:** the app shows the method. It is a demonstration and not a tool for real decisions about real transactions.
- **Very small group:** the 3-flag group has only 7 transactions, so its 100% fraud rate is not a reliable rule.
- **Assumed costs:** the threshold of 0.5 assumes that a missed fraud costs more than a false alarm (Section 5.5).

B.5 Files and hosting

The code is in the folder **streamlit_app** of the GitHub repository: app.py (the app), final_model.joblib (a copy of the final model), requirements.txt (the Python packages) and README.md. The app is hosted on Streamlit Community Cloud. The same scikit-learn version that saved the model (1.7.2) is used to load it.